
Real-World PBTs as Lean Challenges

Anonymous Author(s)

Affiliation

Address

email

Abstract

We present a benchmark for evaluating AI models on real-world formal software verification tasks. We first scrape $\approx 20k$ property-based tests (PBTs) from permissively licensed real-world Python repositories, then automatically translate them into Lean 4 specifications with sorry placeholders. Translating PBTs into Lean specifications is challenging: it requires modeling Python semantics in Lean, inferring the logical property encoded in an imperative PBT, and handling the inherent difficulties of dependently-typed programming in a seldom-used language. We describe a three-agent LLM pipeline for transpiling PBT suites into Lean specifications, evaluate coverage and quality metrics, and provide baselines for proof completion using several automated and model-based approaches. Our benchmark aims to drive progress on the underexplored problem of AI-assisted formal verification of real-world software, which we claim is of increasing import as AI produces more and more of the world’s code.

1 Introduction

Several AI safety proposals are premised on implementing safeguards by way of *formal verification* (FV)¹ – that is, software systems that can produce and unequivocally certify mathematical proofs. These systems can be split into *interactive theorem provers* (ITPs) which accept and validate user-supplied proofs, and *automated theorem provers* (ATPs) which attempt (but often fail) to prove theorems fully autonomously. ARIA Safeguarded AI [9], Guaranteed Safe AI [8], and Scalable Formal Oversight [30, 22], among others, all suggest that in the future, all AI-generated software will be formally verified by autonomous FV agents, who will steer ITPs in much the same way that coding agents currently steer software production.

The only way FV can play this role is if agents can conduct verification work at least as well and as quickly as they can produce code. Otherwise, FV becomes a bottleneck and will naturally wither under the capital pressure to iterate². There is currently little evidence that such capabilities are possible. To achieve them, we would first need high-quality benchmarks to train on. Existing benchmarks (see Table 1) are too small [19], focused on advanced mathematics rather than engineering [11], or derived from expert-driven verification projects [6] which are quite different from what you would get were you to verify the average software project. To clarify this last point: historically, FV has been bottle-necked by the number of human expert practitioners able to write proofs in an ITP³, and therefore, FV was only ever applied to the most mathematically complex and safety-critical

¹FV forms a subset of *formal methods*, which also include PBT, model checking, and abstract interpretation.

²The reader is referred to [25] for a detailed discussion of how socioeconomic pressures degrade safety cultures and cause predictable accidents.

³We were unable to find any formal estimate of the number of human formal methods practitioners today. However, as a very rough proxy, de Almeida Borges et al. [10] conducted the largest ever survey of users of any particular ITP – in this case Coq, which is now called Rocq – at 466 participants. As a point of contrast, that year’s survey by the Rust foundation of Rust software engineers had 7,156 participants [27]. Anecdotally, most ITP work today is in Rocq, Lean 4, or Isabelle.

applications, such as compilers [16] or an OS kernel [15]. These projects are generally monolithic, closed systems, in contrast to the average software project which may have a frontend and backend, a database, reliance on several external black-box APIs such as an inference provider or a logging service, and an unnecessarily large dependency tree. The latter category of (normal) software is actually in many ways much more challenging to verify because it necessitates reasoning about black-box external systems and the interactions of several software systems, potentially written in different languages and operating on different chips [4, 23].

Benchmark	ITP(s)	# theorems	Focus
Minif2f [34]	Metamath, Lean, Isabelle, HOL Light	488	math Olympiad
PutnamBench [29]	Lean, Isabelle, Coq/Rocq	640	math Olympiad
Fimo [18]	Lean	149	math Olympiad
DafnyBench [19]	Dafny	782	scraped Dafny proofs
ProofNet [1]	Lean	371	undergraduate math
RLM25 [24]	Lean	619	graduate math
Lean Workbook [32]	Lean	57,000	high-school through graduate math
FormalMATH [33]	Lean	5,560	math Olympiad
LEAN-GitHub [31]	Lean	28,597	scraped Lean proofs
DeepSeek-ProverBench [26]	Lean	325	math Olympiad
MiniCTX [12]	Lean	762	six open-source Lean projects
LeanGeo [7]	Lean	122	high-school through graduate math
IneqMath [21]	Lean	1,522	math Olympiad
FATE- $\{M,H,X\}$ [14]	Lean	350	undergraduate through graduate math
VeriBench [3]	Lean	857	software verification
CLEVER [28]	Lean	161	software synthesis and verification
Vericoding [5]	Lean, Verus, Dafny	12,504	software synthesis and verification
MUSTARD/MUSTARDSAUCE [13]	Lean	5,866	high-school through undergraduate math
FormL4 [20]	Lean 4	14,510	undergraduate math
CoqStoq (Rango) [?]]	Coq	10,396	software verification
MSC-180 [17]	Lean	180	undergraduate through graduate math
	HOL Light	29,462	undergraduate through graduate math

Table 1: Prior ITP benchmarks. If a benchmark improved upon a prior benchmark, we excluded the prior one.

References

- [1] Z. Azerbayev, B. Piotrowski, H. Schoelkopf, E. W. Ayers, D. Radev, and J. Avigad. Proofnet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023.
- [2] K. Bansal, S. Loos, M. Rabe, C. Szegedy, and S. Wilcox. Holist: An environment for machine learning of higher order logic theorem proving. In *International conference on machine learning*, pages 454–463. PMLR, 2019.
- [3] S. Barkallah, S. Daruru, B. Miranda, L. Aniva, A. Nie, and S. Koyejo. Veribench-ftp: A formal theorem proving benchmark in lean 4 for code verification. In *The 5th Workshop on Mathematical Reasoning and AI at NeurIPS 2025*.
- [4] S. Buro, R. Crole, and I. Mastroeni. Equational logic and categorical semantics for multi-languages. *Electronic Notes in Theoretical Computer Science*, 352:79–103, 2020.

- [5] S. Bursuc, T. Ehrenborg, S. Lin, L. Astefanoaei, I. E. Chiosa, J. Kukovec, A. Singh, O. Butterley, A. Bizid, Q. Dougherty, et al. A benchmark for vericoding: formally verified program synthesis. *arXiv preprint arXiv:2509.22908*, 2025.
- [6] S. Chakraborty, G. Ebner, S. Bhat, S. Fakhoury, S. Fatima, S. Lahiri, and N. Swamy. Towards neural synthesis for SMT-assisted proof-oriented programming. In *Proceedings of the 47th International Conference on Software Engineering (ICSE)*, 2025.
- [7] F. P. H. W. X. L. J. L. J. L. Z. L. Chendong Song, Zihan Wang. LeanGeo: Formalizing computational geometry problems in Lean. *arXiv preprint arXiv:2508.14644v1*, 2025.
- [8] D. Dalrymple, J. Skalse, Y. Bengio, S. Russell, M. Tegmark, S. Seshia, S. Omohundro, C. Szegedy, B. Goldhaber, N. Ammann, A. Abate, J. Halpern, C. Barrett, D. Zhao, T. Zhi-Xuan, J. Wing, and J. Tenenbaum. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems. *arXiv preprint arXiv:2405.06624*, 2024.
- [9] N. A. David Dalrymple. Safeguarded ai. <https://aria.org.uk/opportunity-spaces/mathematics-for-safe-ai/safeguarded-ai/>, 2025. Accessed 25 April 2026.
- [10] A. de Almeida Borges, A. Casanueva Artís, J.-R. Falleri, E. J. Gallego Arias, É. Martin-Dorel, K. Palmiskog, A. Serebrenik, and T. Zimmermann. Lessons for interactive theorem proving researchers from a survey of coq users. *Journal of Automated Reasoning*, 69(1):8, 2025.
- [11] E. Glazer, E. Erdil, T. Besiroglu, D. Chicharro, E. Chen, A. Gunning, C. Falkman Olsson, J.-S. Denain, A. Ho, E. de Oliveira Santos, O. Järvinen, M. Barnett, R. Sandler, M. Vrzala, J. Sevilla, Q. Ren, E. Pratt, L. Levine, G. Barkley, N. Stewart, B. Grechuk, T. Grechuk, S. Varma Enugandla, and M. Wildon. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv preprint arXiv:2411.04872*, 2024.
- [12] J. Hu, T. Zhu, and S. Welleck. minictx: Neural theorem proving with (long-) contexts. *arXiv preprint arXiv:2408.03350*, 2024.
- [13] Y. Huang, X. Lin, Z. Liu, Q. Cao, H. Xin, H. Wang, Z. Li, L. Song, and X. Liang. Mustard: Mastering uniform synthesis of theorem and proof data. *arXiv preprint arXiv:2402.08957*, 2024.
- [14] J. Jiang, W. He, Y. Wang, G. Gao, Y. Hu, J. Wang, N. Guan, P. Wu, C. Dai, L. Xiao, and B. Dong. Fate: A formal benchmark series for frontier algebra of multiple difficulty levels. *arXiv preprint arXiv:2511.02872*, 2025.
- [15] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, et al. sel4: Formal verification of an os kernel. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, pages 207–220, 2009.
- [16] X. Leroy, S. Blazy, D. Kästner, B. Schommer, M. Pister, and C. Ferdinand. CompCert-a formally verified optimizing compiler. In *ERTS 2016: Embedded Real Time Software and Systems, 8th European Congress*, 2016.
- [17] S. Li, W. Lu, X. Shi, K. Weng, H. Sun, M. Yu, T. Zhang, G. Yu, H. Liu, and L. Du. MSC-180: A benchmark for automated formal theorem proving from mathematical subject classification. *arXiv preprint arXiv:2512.18256*, 2025.
- [18] C. Liu, J. Shen, H. Xin, Z. Liu, Y. Yuan, H. Wang, W. Ju, C. Zheng, Y. Yin, L. Li, et al. Fimo: A challenge formal dataset for automated theorem proving. *arXiv preprint arXiv:2309.04295*, 2023.
- [19] C. Loughridge, Q. Sun, S. Ahrenbach, F. Cassano, C. Sun, Y. Sheng, A. Mudide, M. R. H. Misu, N. Amin, and M. Tegmark. DafnyBench: A benchmark for formal software verification. *arXiv preprint arXiv:2406.08467*, 2024.
- [20] J. Lu, Y. Wan, Z. Liu, Y. Huang, J. Xiong, C. Liu, J. Shen, H. Jin, J. Zhang, H. Wang, et al. Process-driven autoformalization in lean 4. *arXiv preprint arXiv:2406.01940*, 2024.

- [21] P. Lu, J. Sheng, L. Lyu, J. Jin, T. Xia, A. Gu, and J. Zou. Solving inequality proofs with large language models. In *The 39th Conference on Neural Information Processing Systems (NeurIPS)*, 2025.
- [22] E. Miyazono. Formally scalable ai oversight through specifications. <https://foresight.org/resource/evan-miyazono-formally-scalable-ai-oversight-through-specifications/>, 2024. Accessed 25 April 2026.
- [23] J. T. Perconti and A. Ahmed. Verifying an open compiler using multi-language semantics. In *European Symposium on Programming Languages and Systems*, pages 128–148. Springer, 2014.
- [24] A. Poiroux, G. Weiss, V. Kunčák, and A. Bosselut. Reliable evaluation and benchmarks for statement autoformalization. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 17958–17980, 2025.
- [25] J. Rasmussen. Risk management in a dynamic society: a modelling problem. *Safety science*, 27(2-3):183–213, 1997.
- [26] Z. Ren, Z. Shao, J. Song, H. Xin, H. Wang, W. Zhao, L. Zhang, Z. Fu, Q. Zhu, D. Yang, et al. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.
- [27] R. S. Team. 2025 state of rust survey results. <https://blog.rust-lang.org/2026/03/02/2025-State-Of-Rust-Survey-results/>, March 2026. Accessed 25 April 2026.
- [28] A. Thakur, J. Lee, G. Tsoukalas, M. Sistla, M. Zhao, S. Zetsche, G. Durrett, Y. Yue, and S. Chaudhuri. Clever: A curated benchmark for formally verified code generation. *arXiv preprint arXiv:2505.13938*, 2025.
- [29] G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition. *Advances in Neural Information Processing Systems*, 37:11545–11569, 2024.
- [30] M. von Hippel. <https://www.lesswrong.com/posts/SfhFh9Hfm6JYvzbby/the-scalable-formal-oversight-research-program>, 2026. Accessed 25 April 2026.
- [31] Z. Wu, J. Wang, D. Lin, and K. Chen. Lean-github: Compiling github lean repositories for a versatile lean prover. *arXiv preprint arXiv:2407.17227*, 2024.
- [32] H. Ying, Z. Wu, Y. Geng, J. Wang, D. Lin, and K. Chen. Lean workbook: A large-scale lean problem set formalized from natural language math problems. *Advances in Neural Information Processing Systems*, 37:105848–105863, 2024.
- [33] Z. Yu, R. Peng, K. Ding, Y. Li, Z. Peng, M. Liu, Y. Zhang, Z. Yuan, H. Xin, W. Huang, et al. Formalmath: Benchmarking formal mathematical reasoning of large language models. *arXiv preprint arXiv:2505.02735*, 2025.
- [34] K. Zheng, J. M. Han, and S. Polu. Minif2f: a cross-system benchmark for formal olympiad-level mathematics. *arXiv preprint arXiv:2109.00110*, 2021.